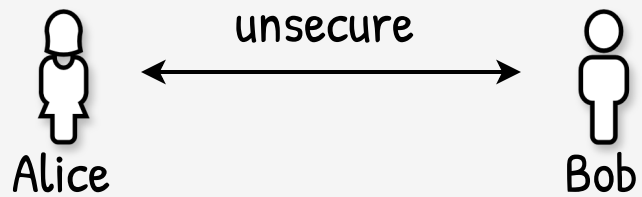
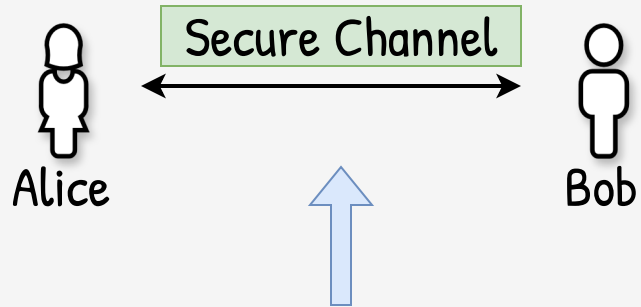


Cryptography II: Asymmetric Crypto, DH, Signatures, Certificates and Web of Trust

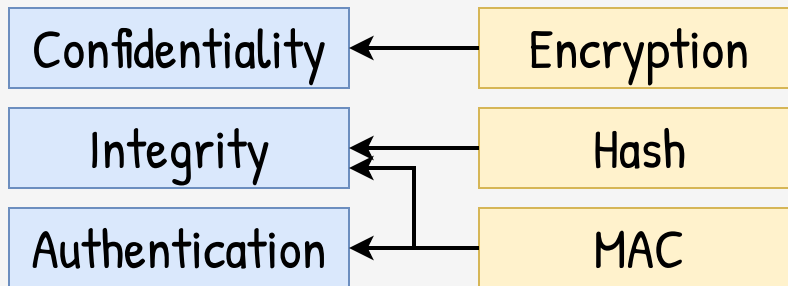
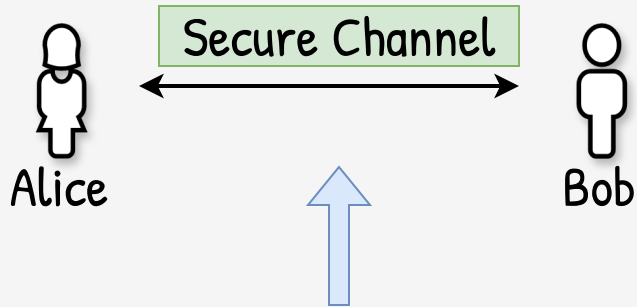
Taller de Seguridad

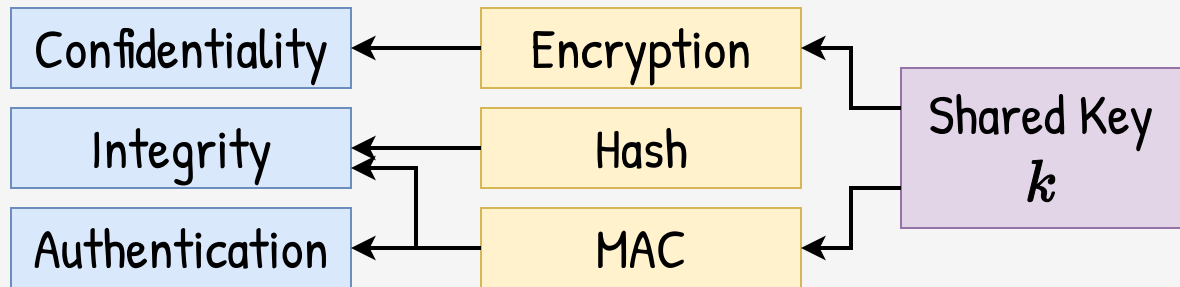
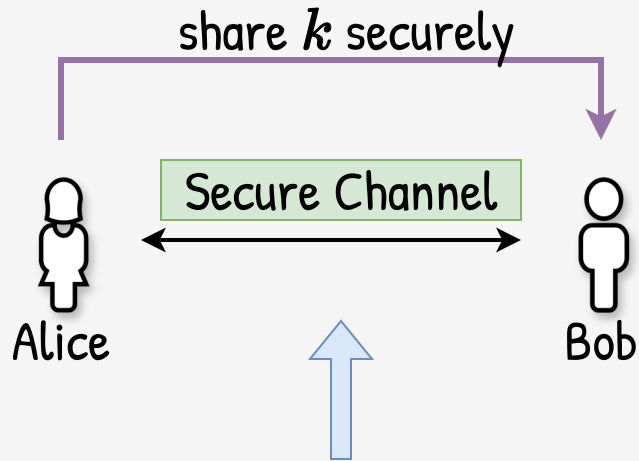
Ing. Martin Di Paola – Fiuba

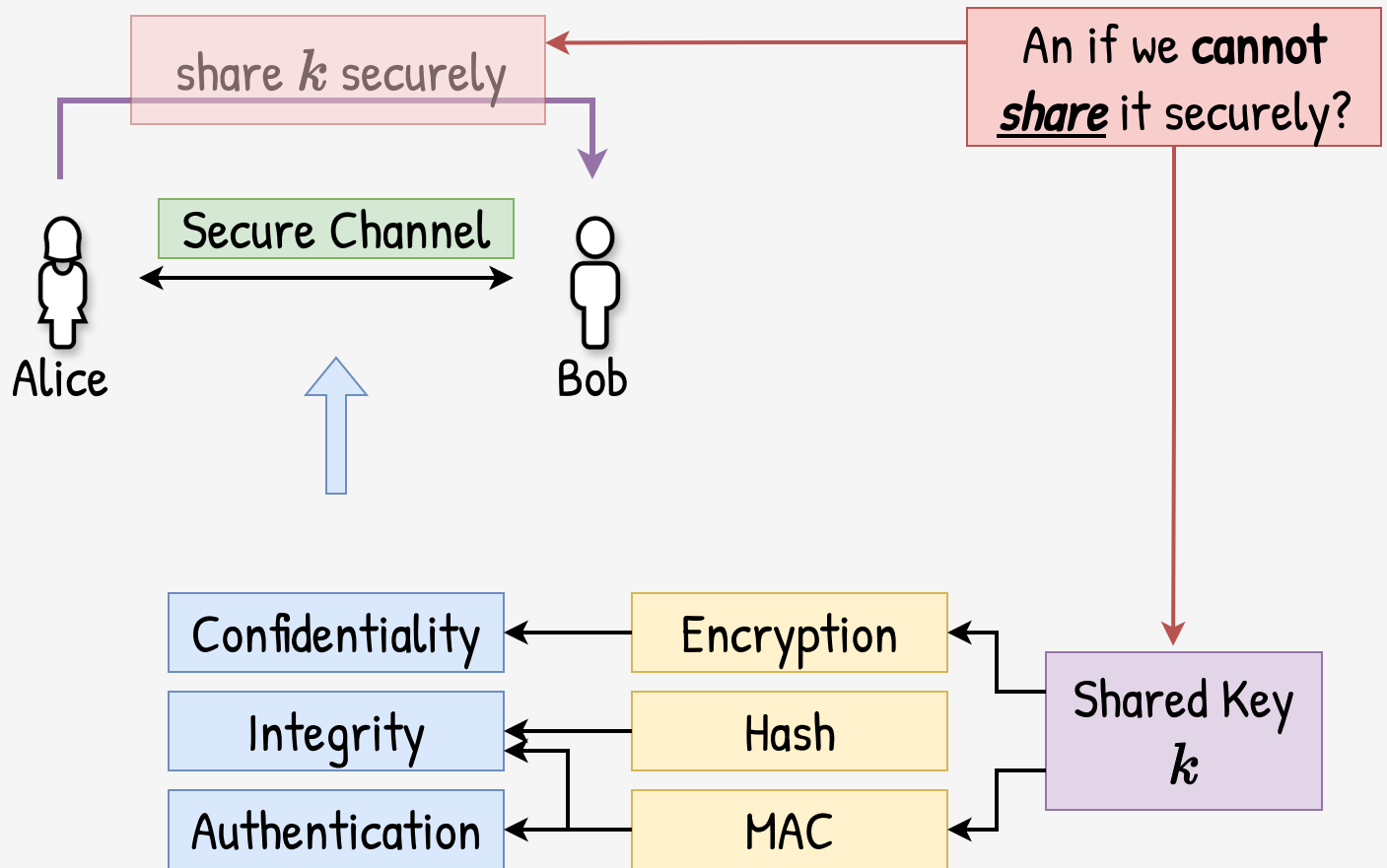


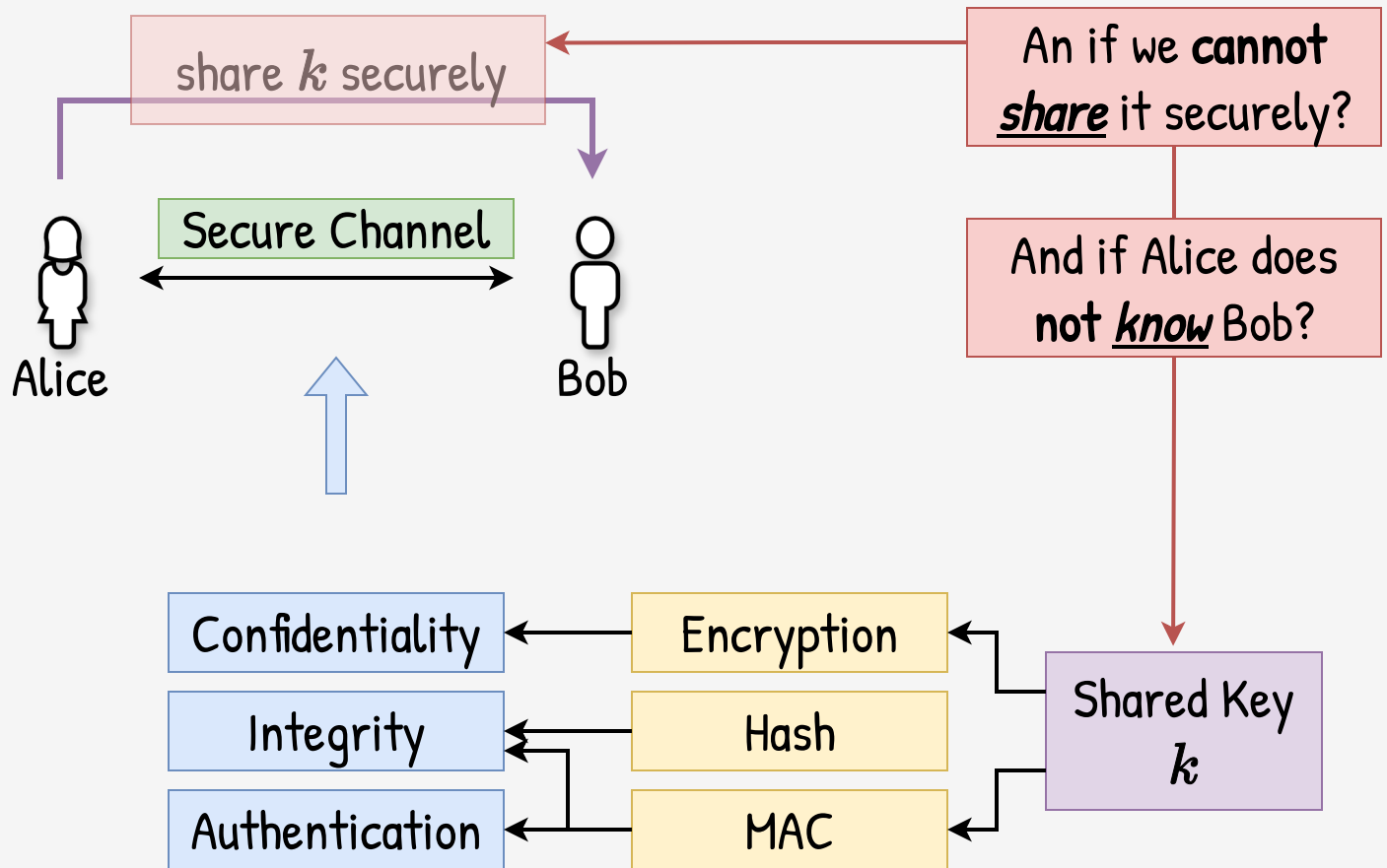


- Confidentiality
- Integrity
- Authentication

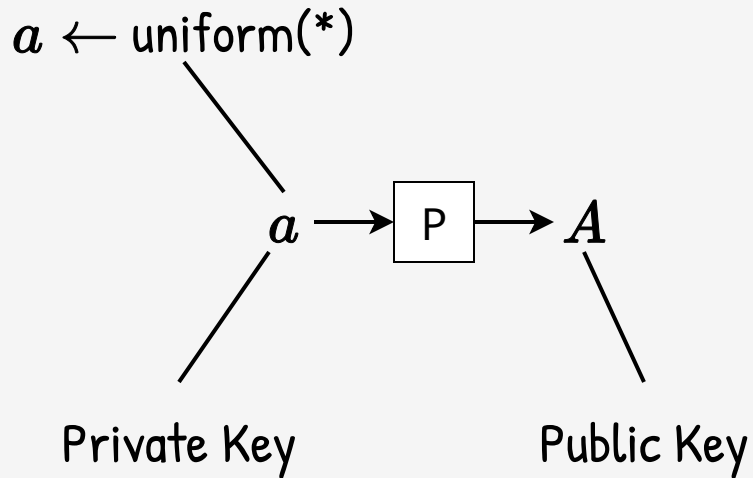




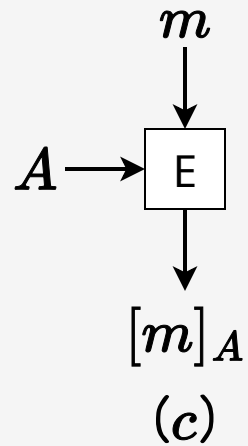
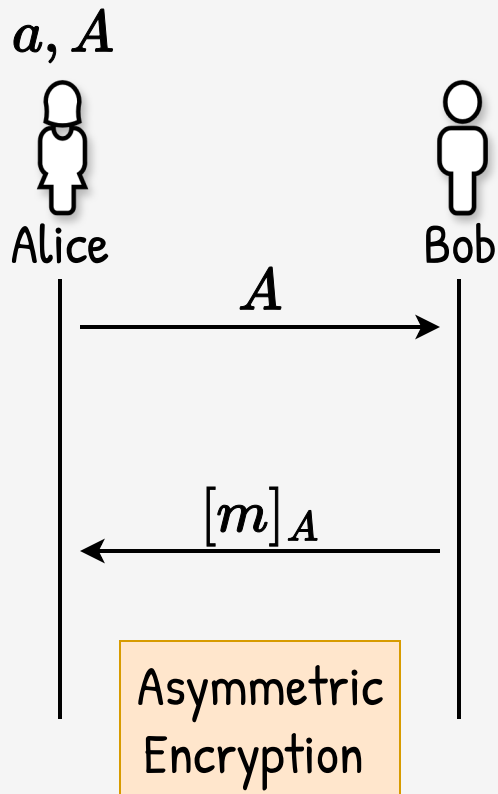
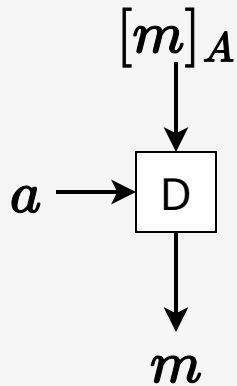




Public Key Derivation



Given A is not
possible to find a



a, A



Alice



Bob

A

$[m]_A$

Asymmetric
Encryption

$[m]_A$

a



m

m

A



$[m]_A$

(c)

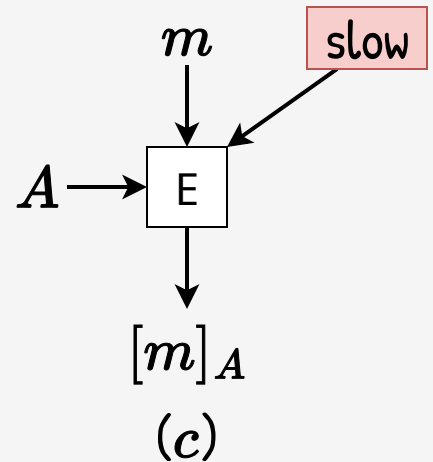
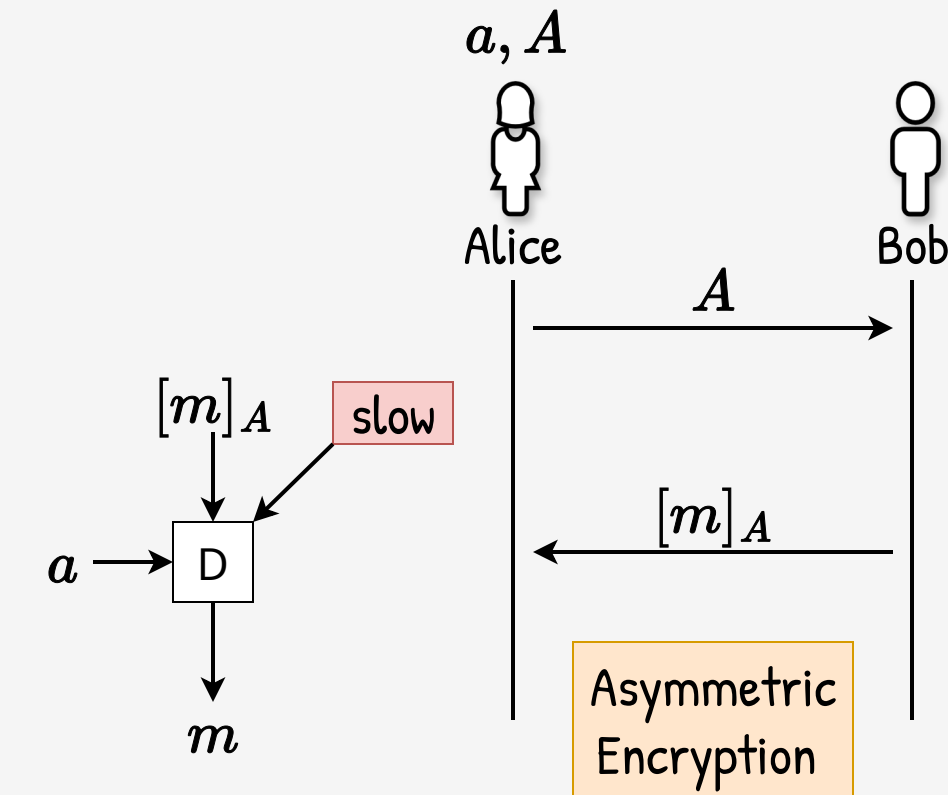
a, A

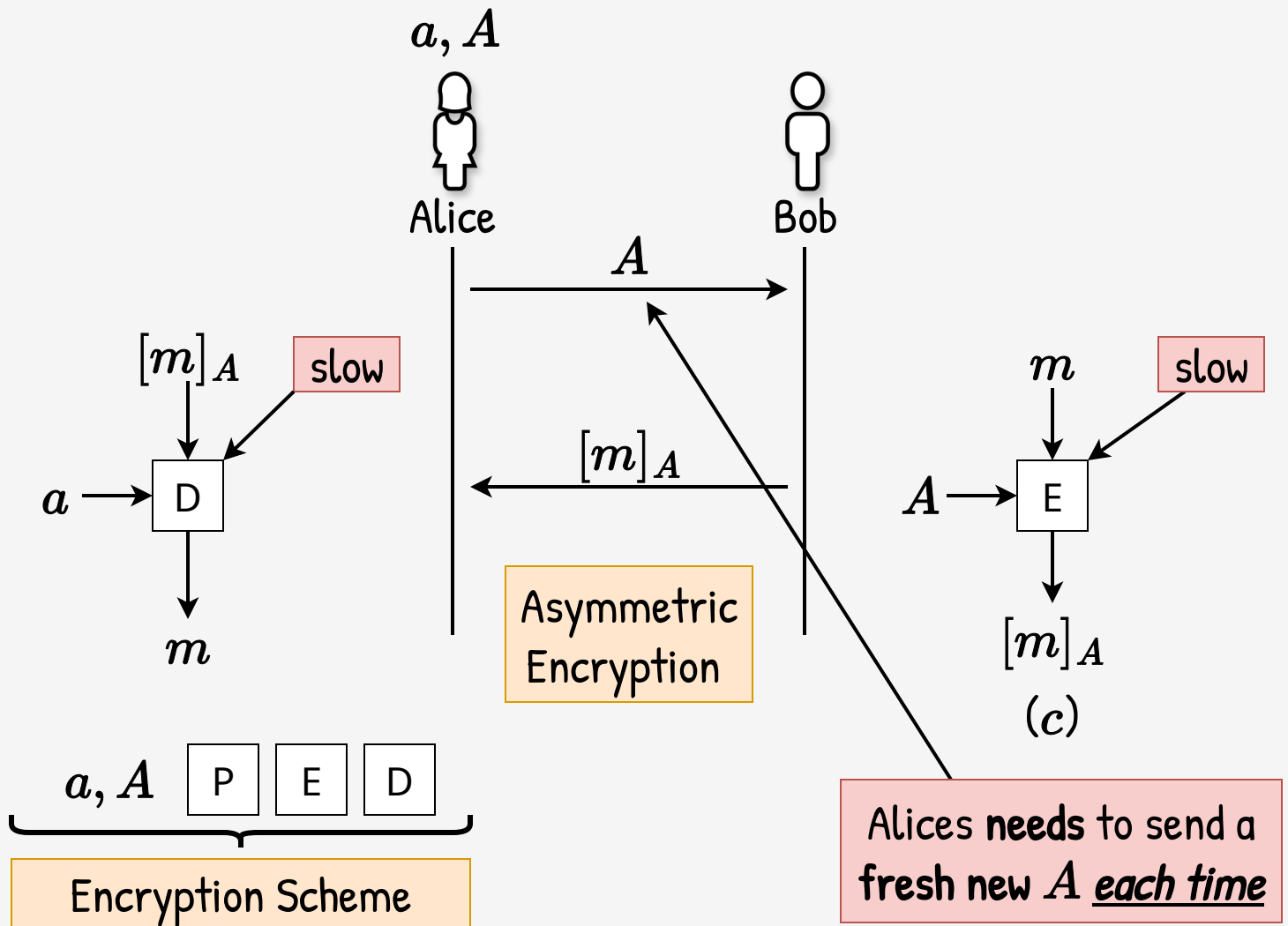
P

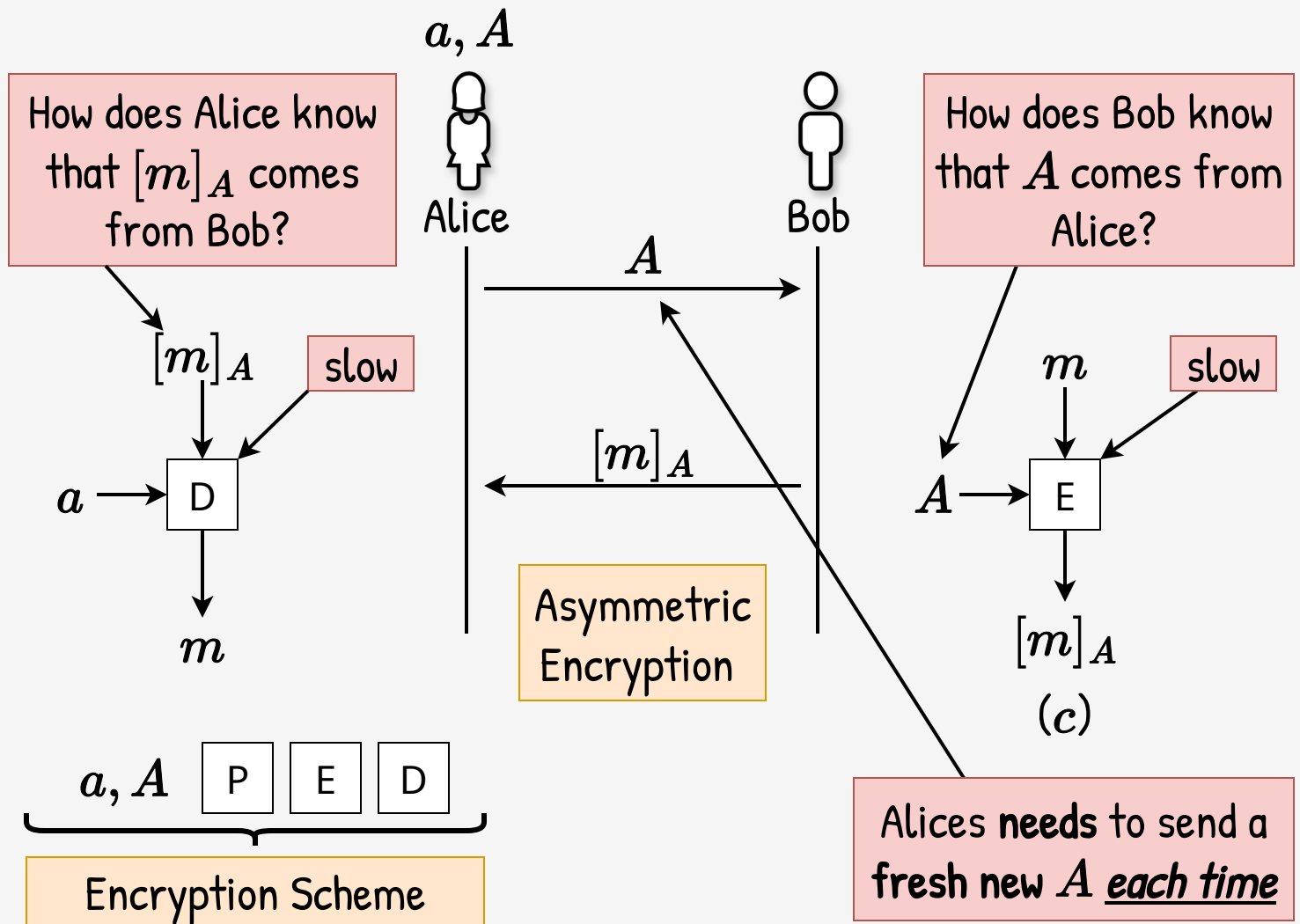
E

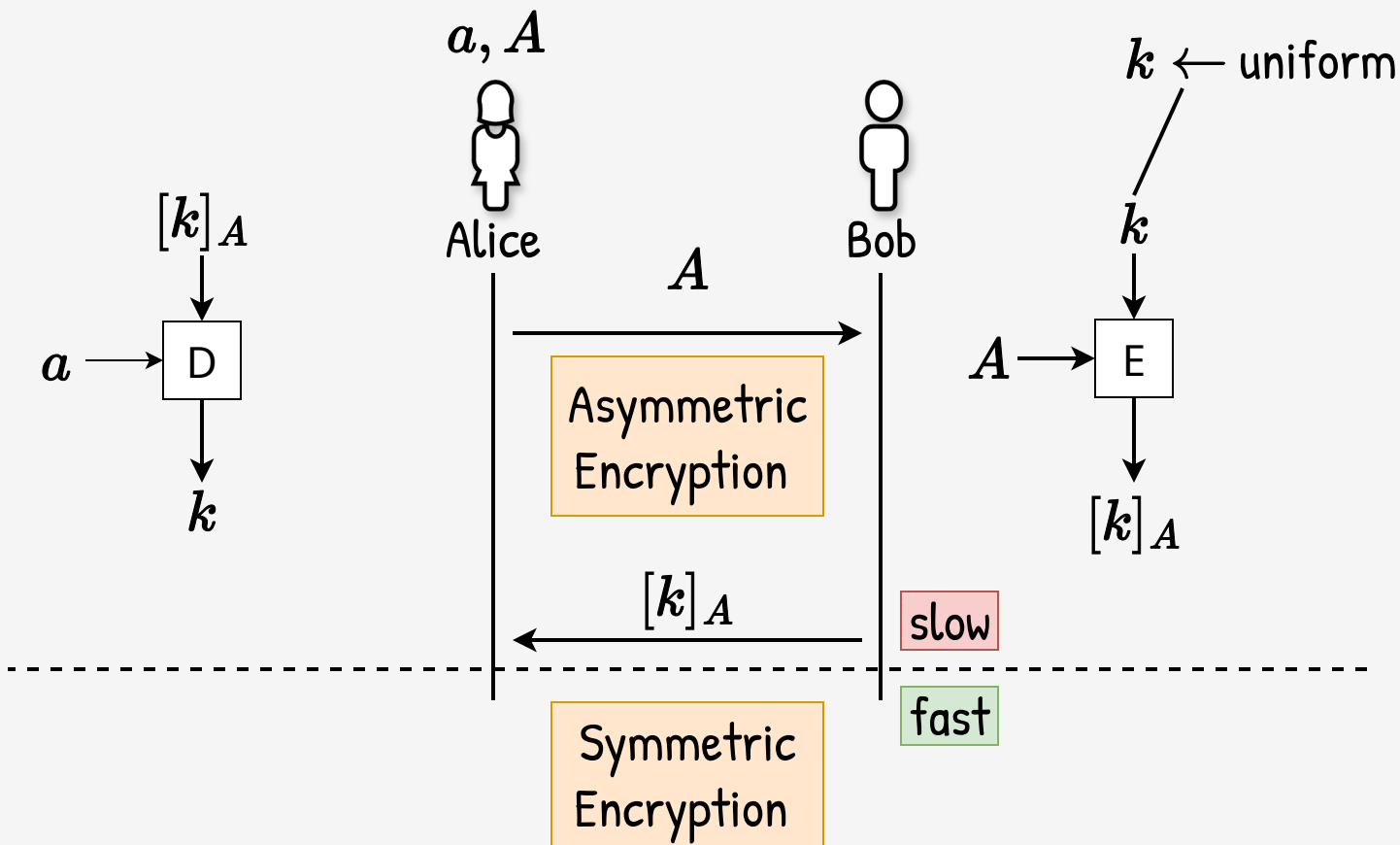
D

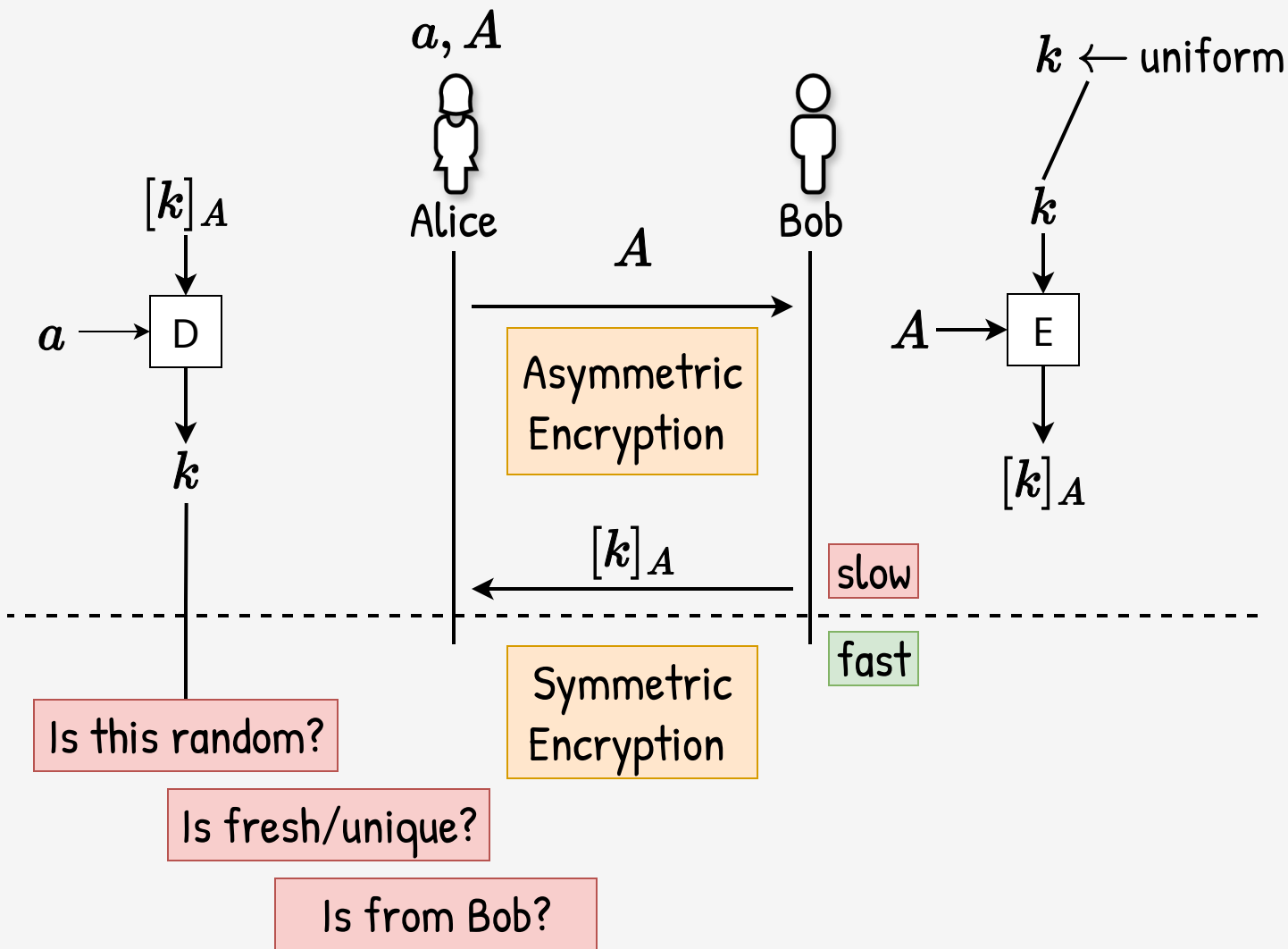
Encryption Scheme











Ephemeral keys for
Key Exchange only.

a_x, A_x

b_x, B_x



Alice



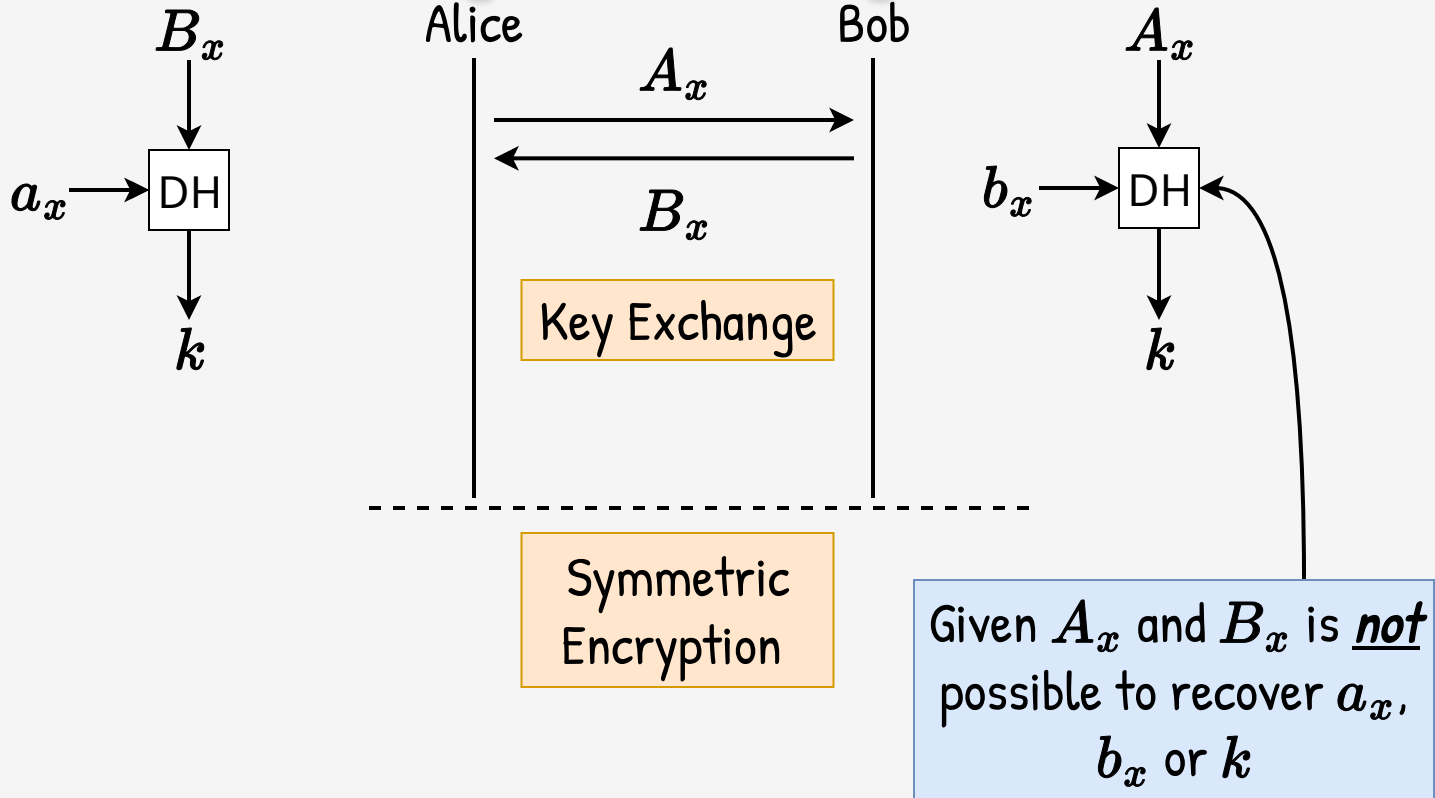
Bob

A_x

B_x

Key Exchange

Ephemeral keys for
Key Exchange only.



Ephemeral keys for
Key Exchange only.

a_x, A_x

b_x, B_x

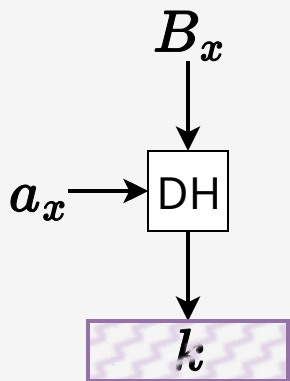
deleted
(after KX)



Alice



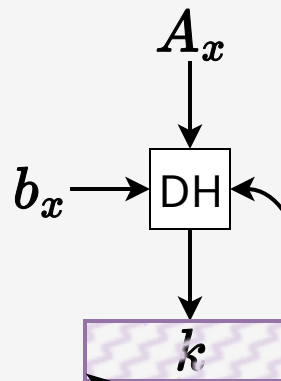
Bob



A_x

B_x

Key Exchange



deleted
(after comm)

Symmetric
Encryption

Forward
Secrecy

Given A_x and B_x is not
possible to recover a_x ,
 b_x or k

$$a \leftarrow \mathbb{N}$$
$$g^a$$

$$(g^b)^a = g^{ab}$$



k

g is publicly
known

Alice

Bob

g^a

g^b

Key Exchange

Diffie-Hellman

$$b \leftarrow \mathbb{N}$$
$$g^b$$

$$(g^a)^b = g^{ab}$$



k

$$a \leftarrow \mathbb{N}$$

$$g^a$$

$$(g^b)^a = g^{ab}$$



k

g is publicly
known



Alice



Bob

g^a

g^b

Key Exchange

Diffie-Hellman

$$b \leftarrow \mathbb{N}$$

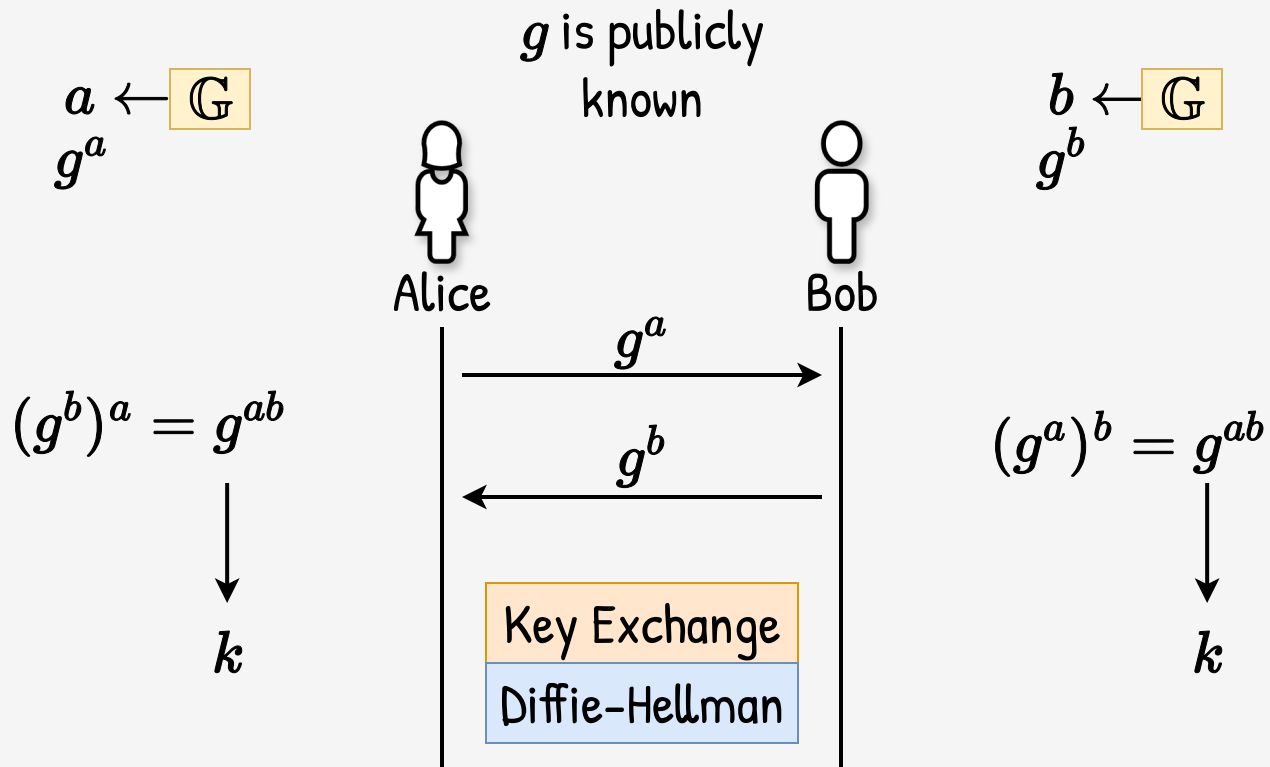
$$g^b$$

$$(g^a)^b = g^{ab}$$



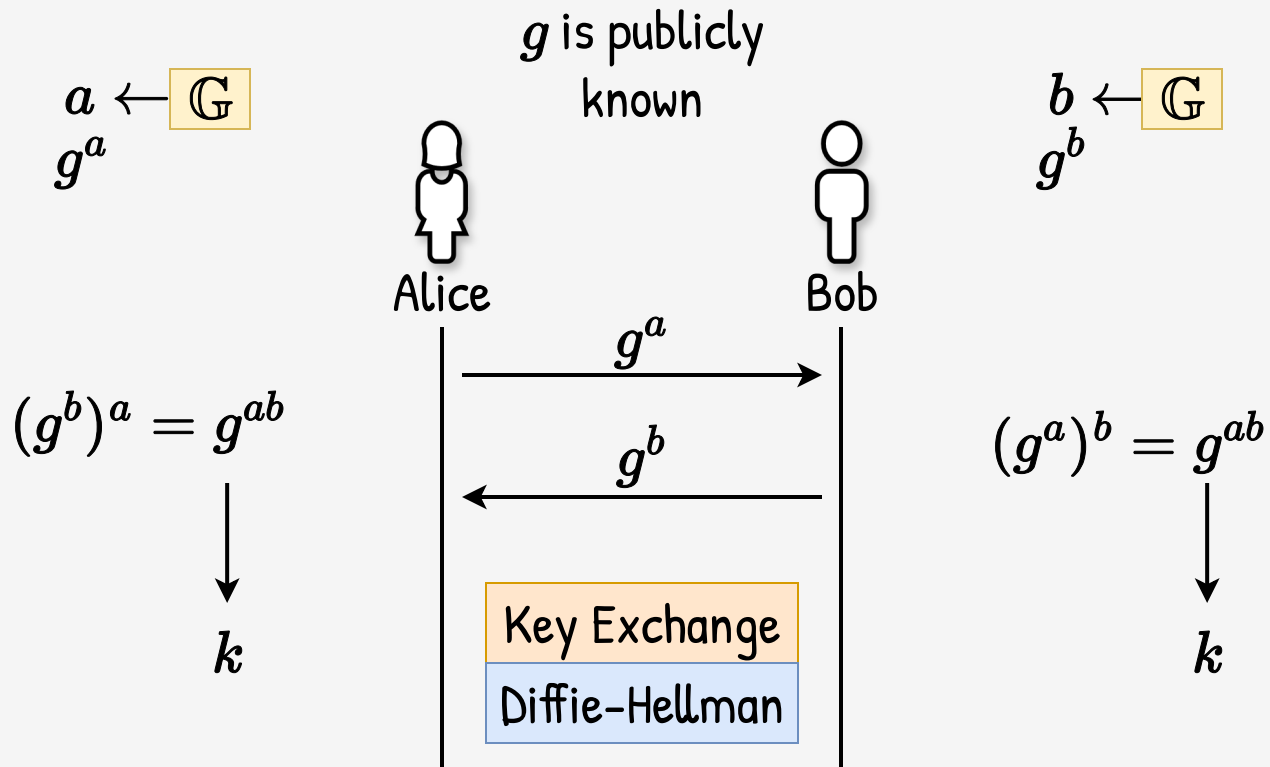
k

But given g^a , it is trivial
to recover a with just
 $\log_g(g^a)$!!



But given g^a , it is trivial
to recover a with just
 $\log_g(g^a)$!!

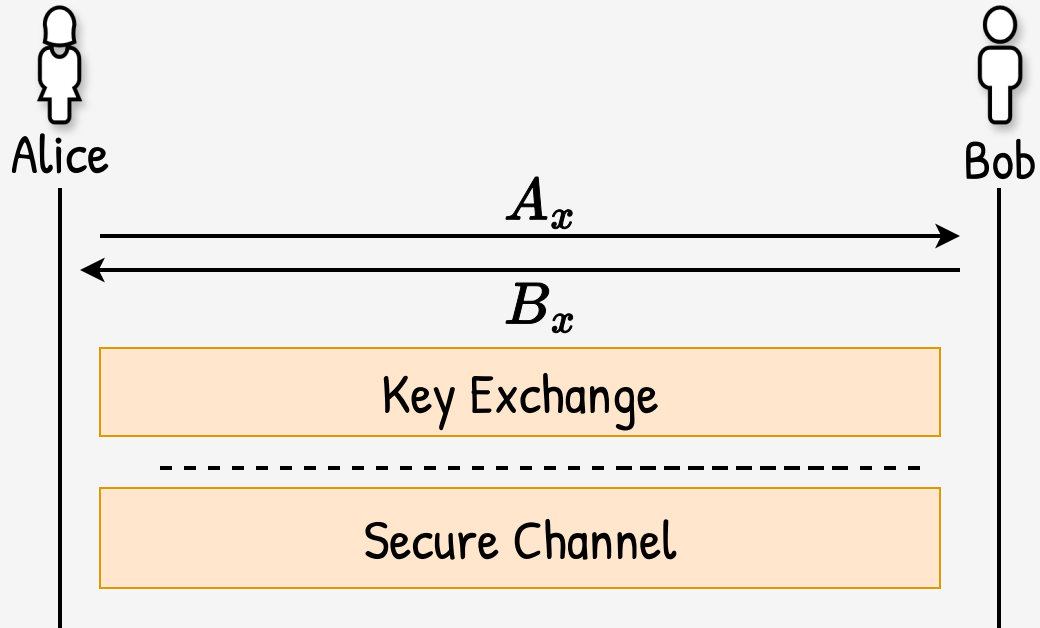
True for $\mathbb{N}$, but the
same math trick works
for **another groups** $\mathbb{G}$

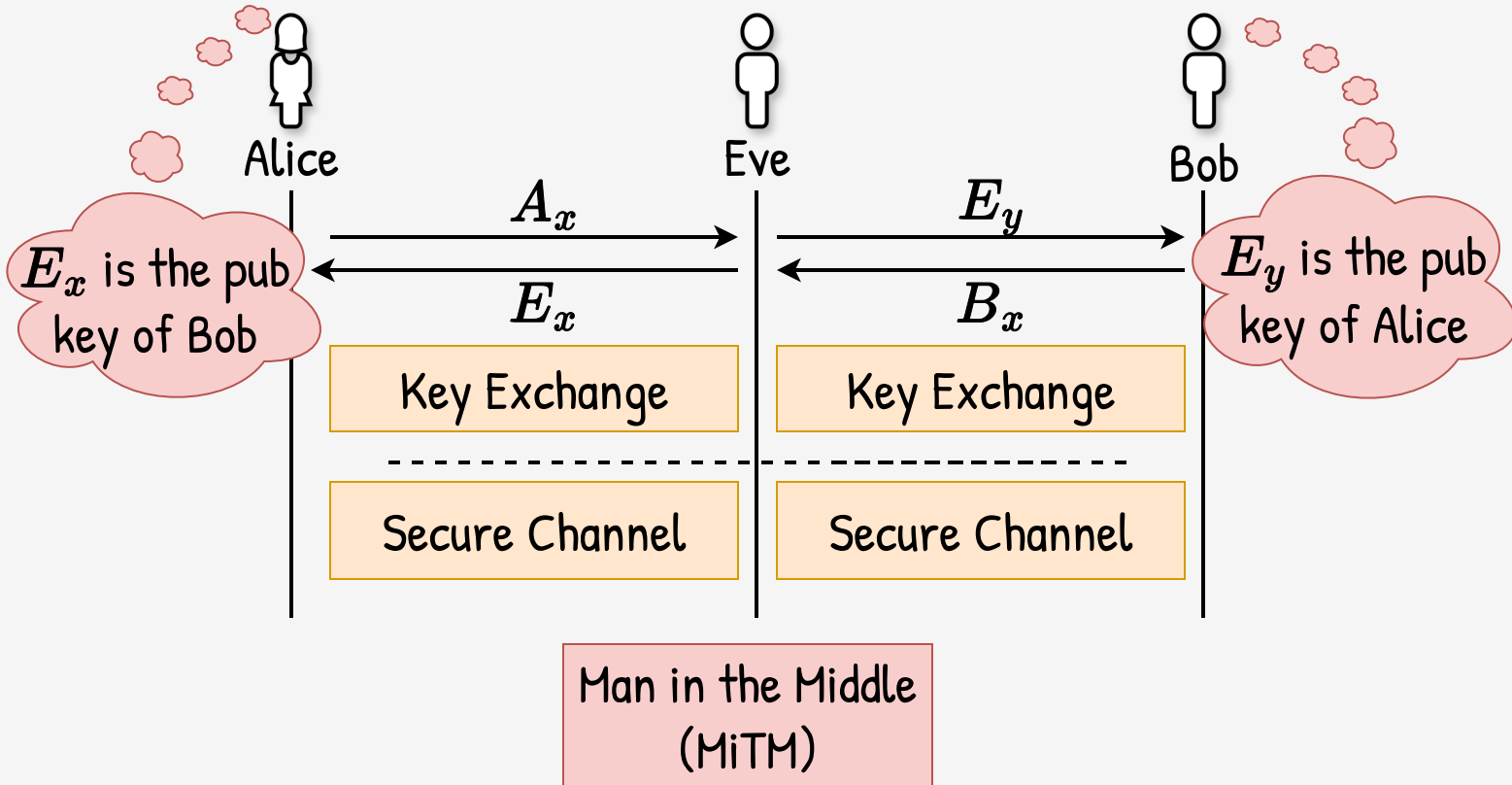


But given g^a , it is trivial
to recover a with just
 $\log_g(g^a)$!!

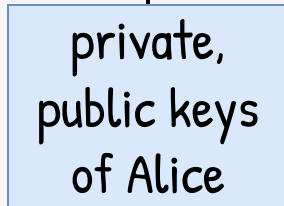
True for $\mathbb{N}$, but the
same math trick works
for **another groups** $\mathbb{G}$

And for some groups $\mathbb{G}$
 $\log_g(g^a)$ is really hard
to compute (for now)



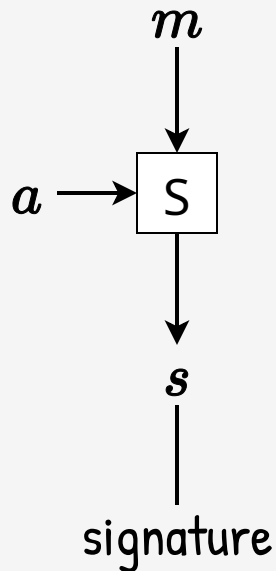


a, A

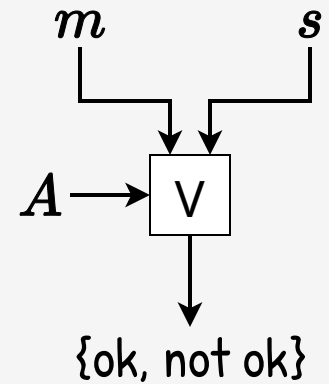


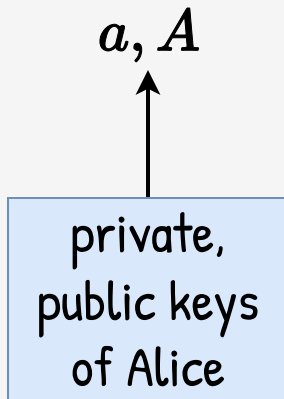
Signature Scheme

Alice
(signer)

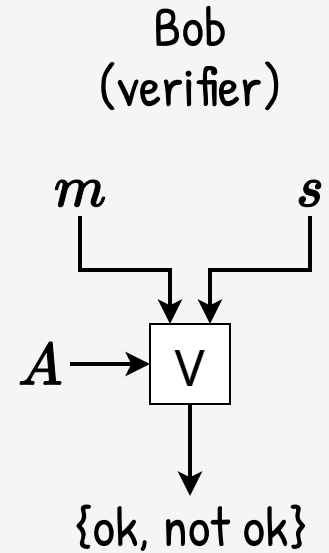
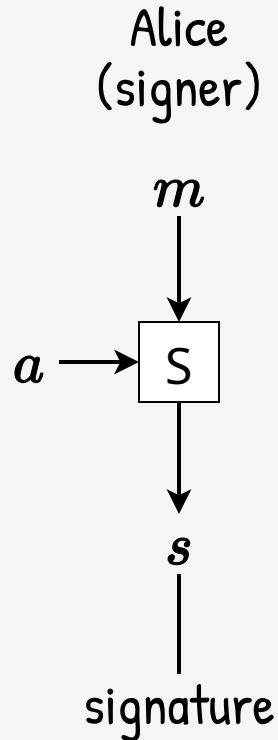


Bob
(verifier)

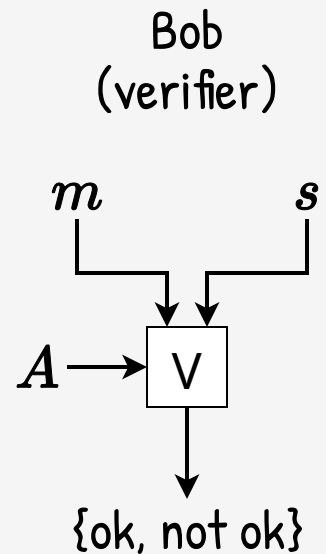
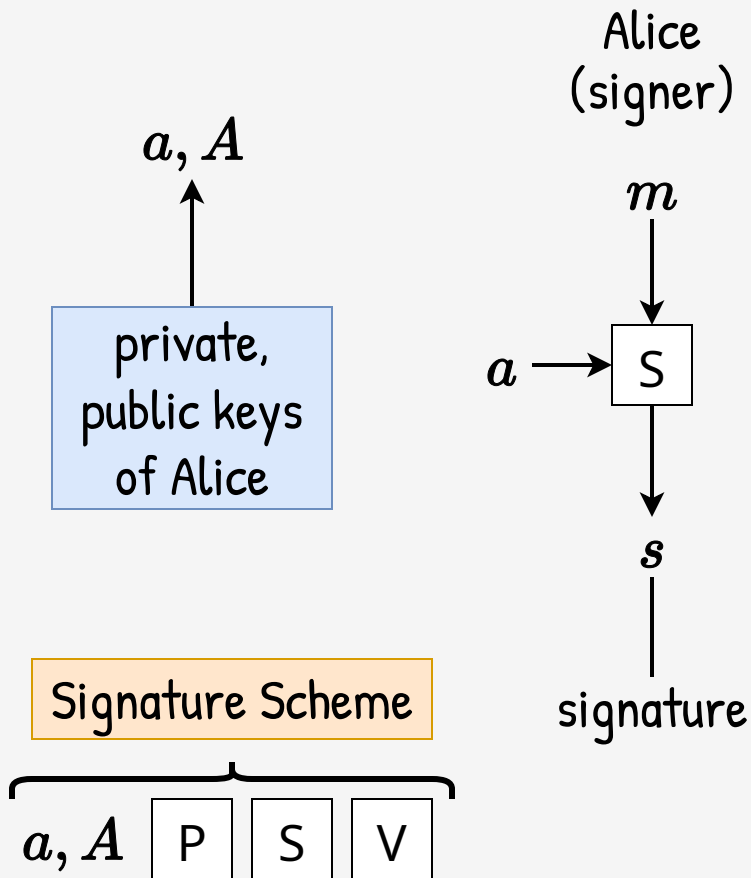




Signature Scheme

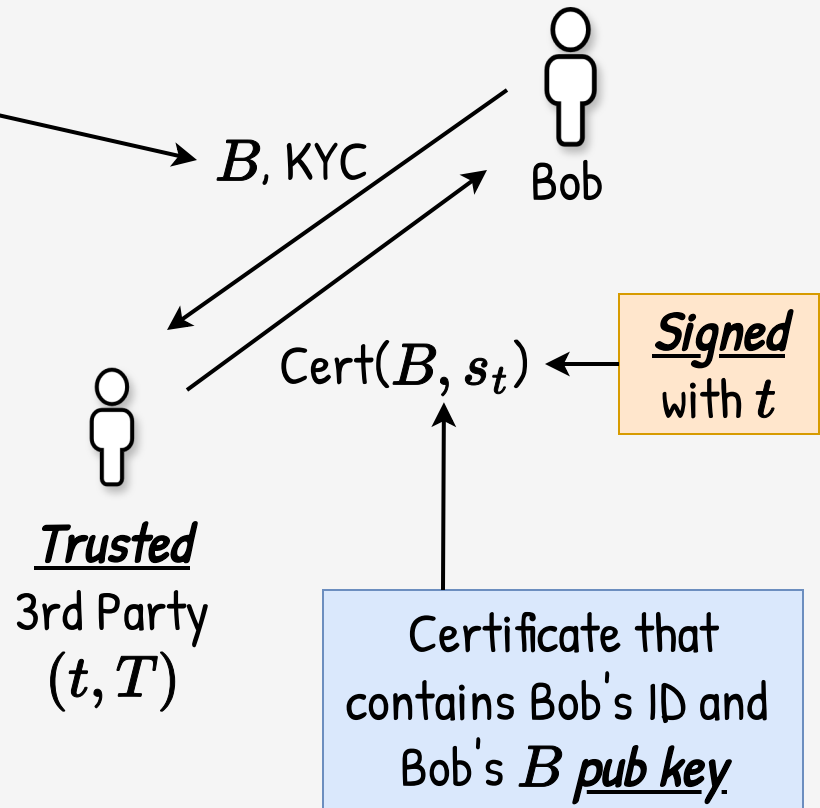


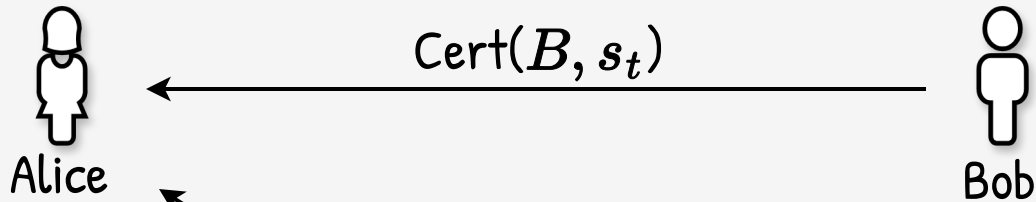
Sign/Verify is slow.
To speed up the op, m is the hash
of m (shorter $\rightarrow$ faster)



Sign/Verify is slow.
To speed up the op, m is the hash
of m (shorter $\rightarrow$ faster)

Certificate Request:
Bob's pub key + KYC to
"prove" Bob is Bob.





$\text{Cert}(T, s_q)$

Is $\text{Cert}(B)$
valid?

- Has B's ID?
- Is s_t valid?
(verified with T)
- Do I trust in the 3rd
Party and its T ?



Trusted
3rd Party
(t, T)

Alice

$\text{Cert}(B, s_t)$

Bob

$\text{Cert}(T, s_q)$

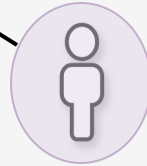
Is $\text{Cert}(B)$
valid?

Has B's ID?

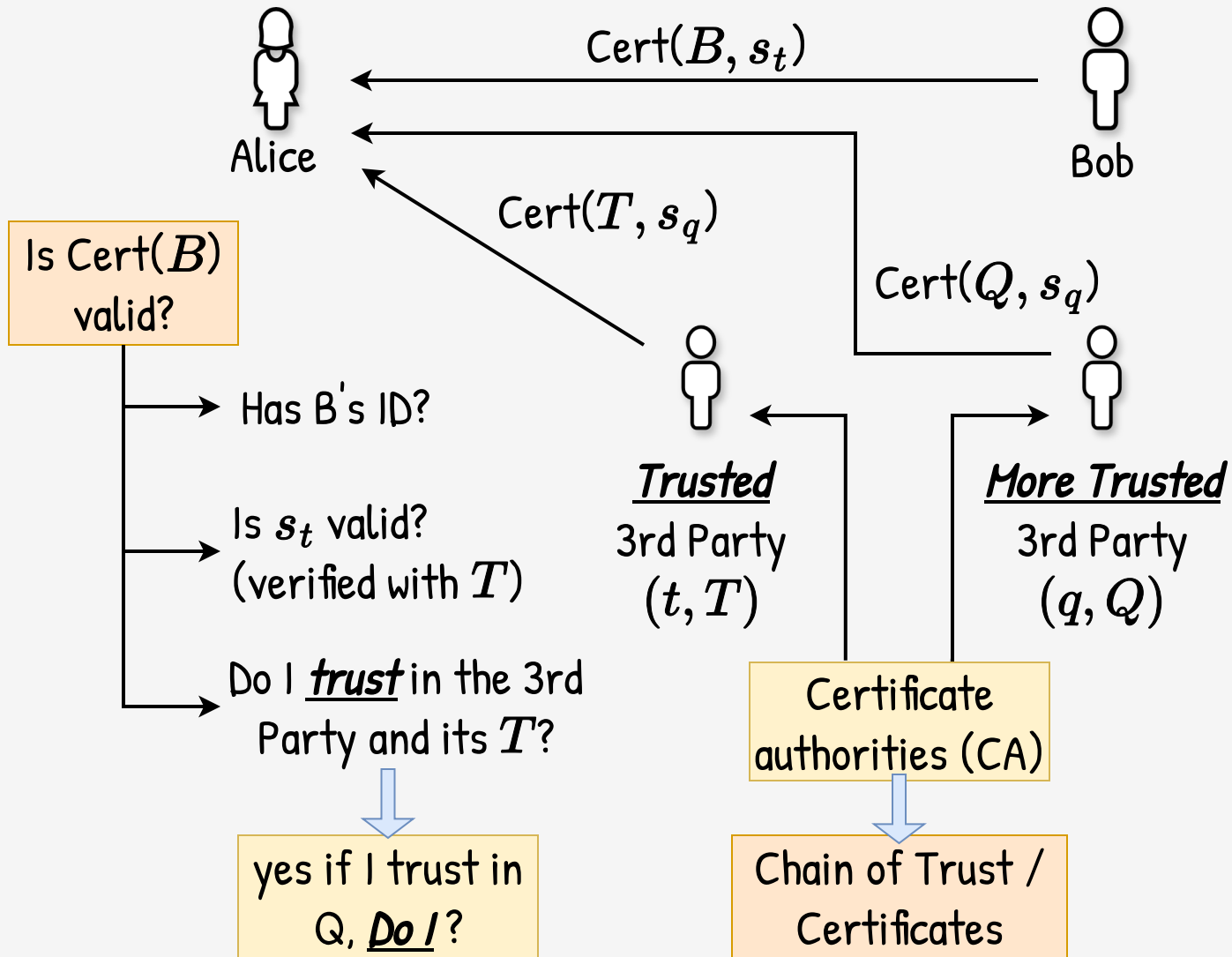
Is s_t valid?
(verified with T)

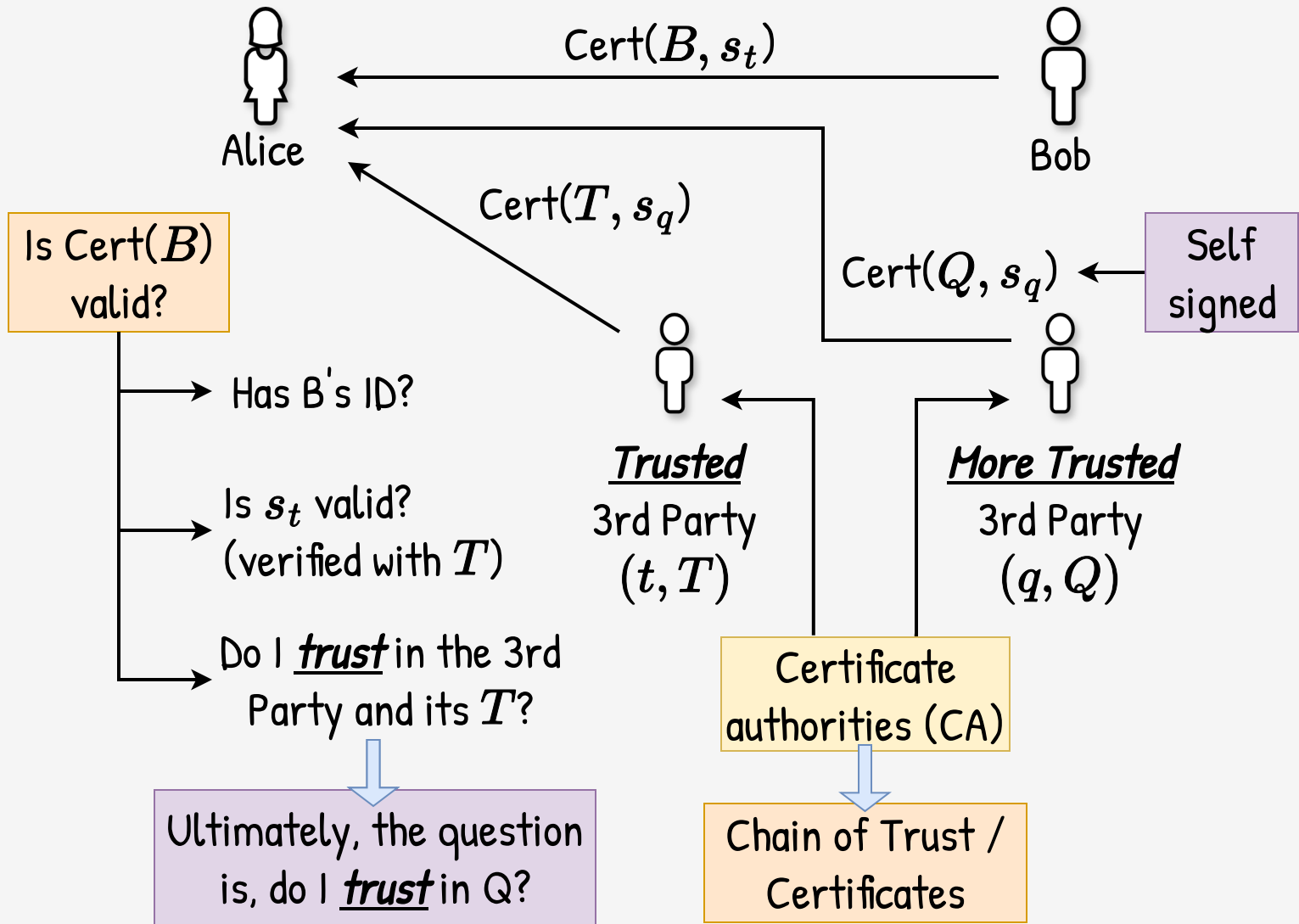
Do I trust in the 3rd
Party and its T ?

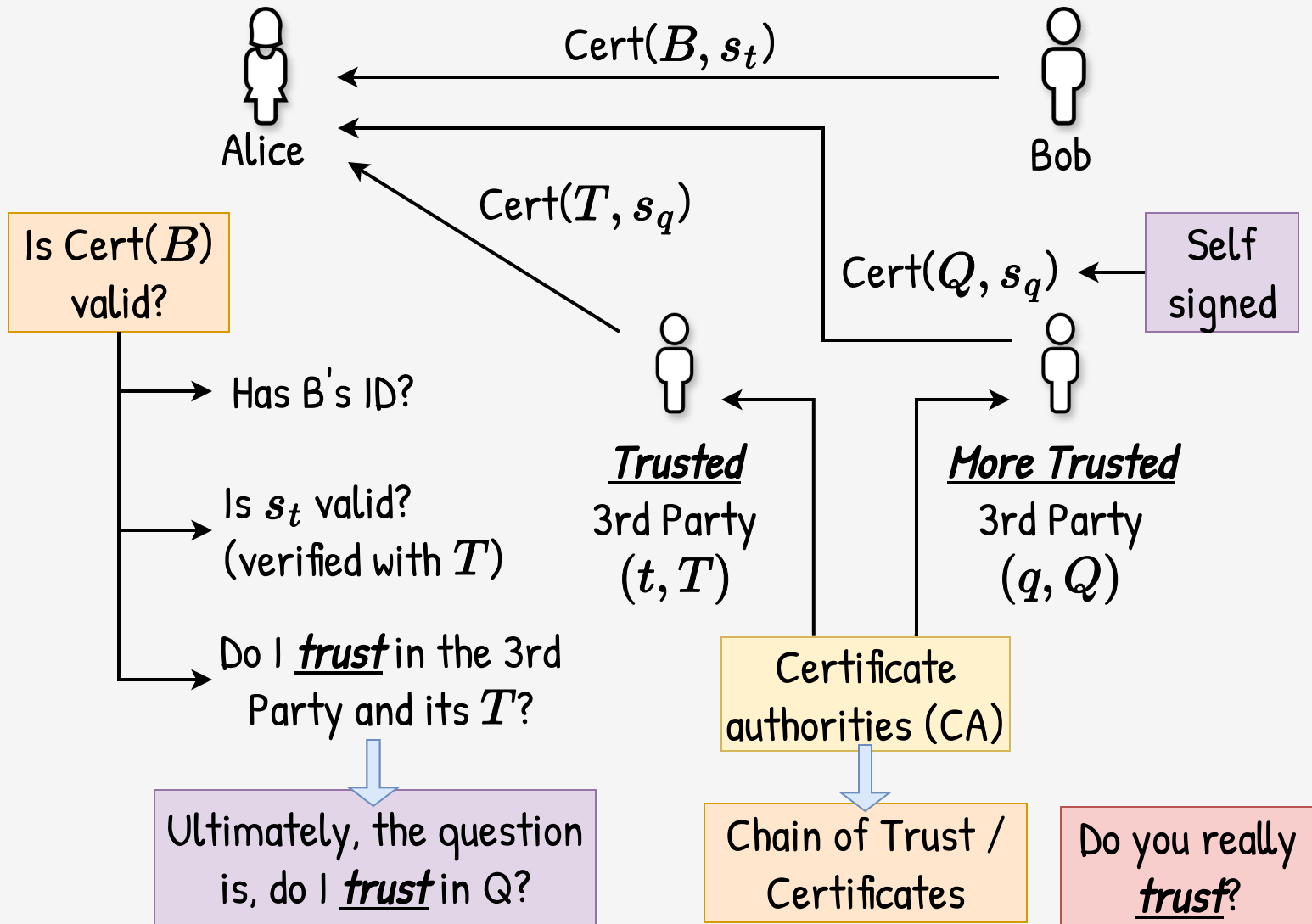
Do you?



Trusted
3rd Party
(t, T)

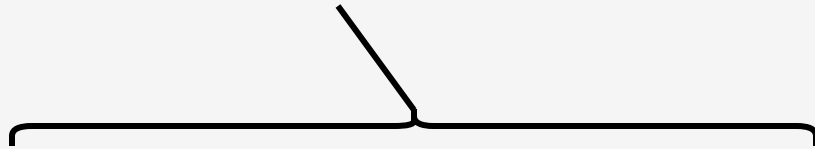








Alice



Owner's

Keys Verified judgement Signed by

A

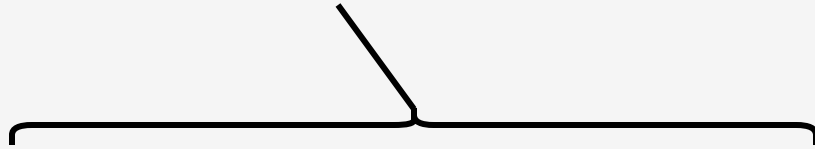
full

full

s_a



Alice



Owner's

Keys Verified judgement Signed by

A

full

full

s_a

B

none

none

s_b

s_d

s_e



Bob



Alice

Owner's

Keys Verified judgement Signed by

A

full

full

s_a

B

none

none

s_b

s_d

s_e



Bob

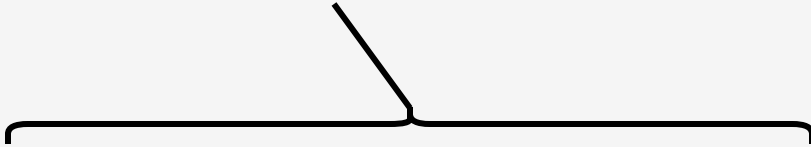
Alice does **not** know, for real,
that *B* belongs to Bob and
has no way to verify it.



Alice



Eric



Owner's

judgement Signed by

Keys

Verified

judgement

Signed by

A

full

full

s_a

B

none

none

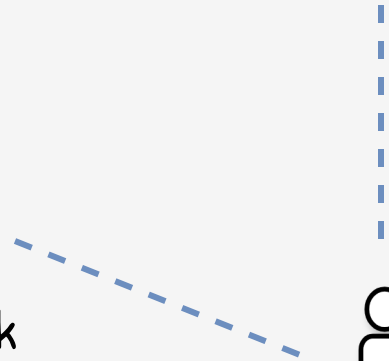
s_b

s_d

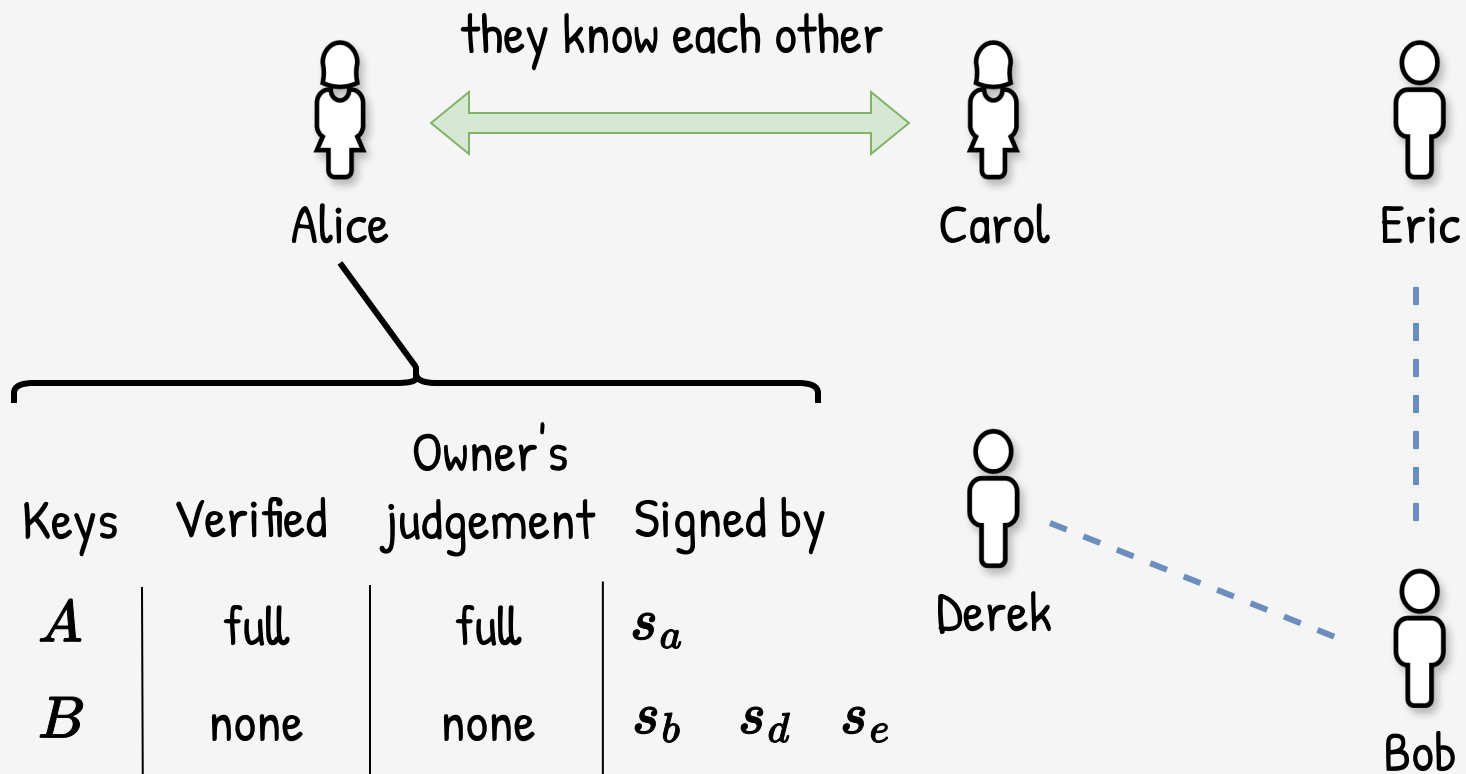
s_e

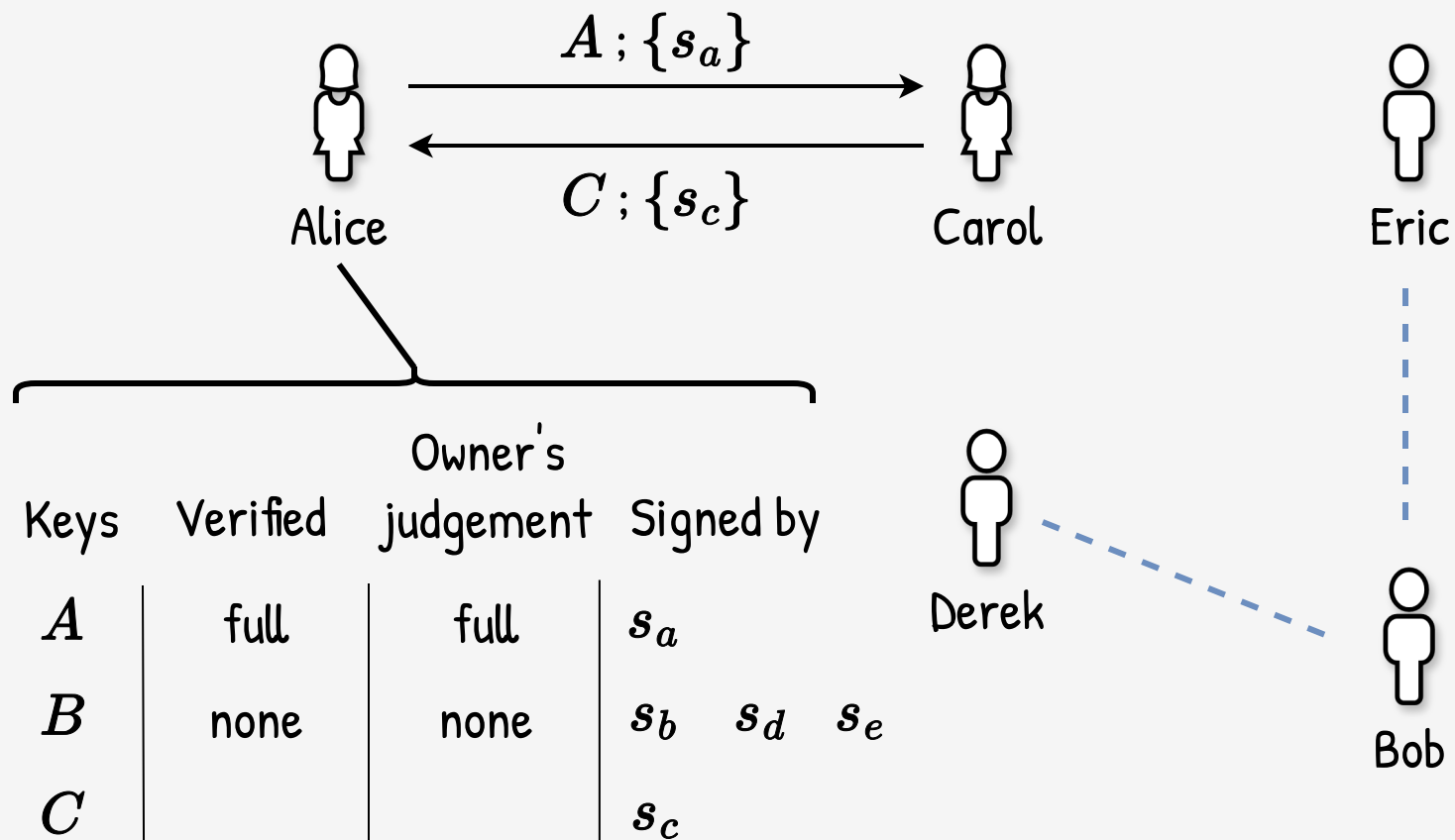


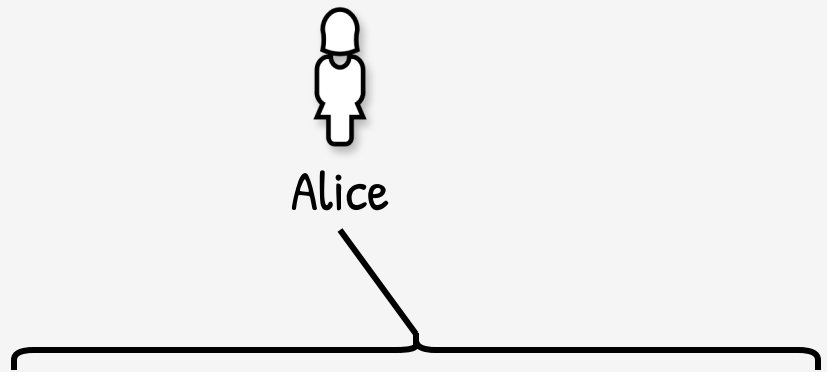
Derek



Bob







Carol

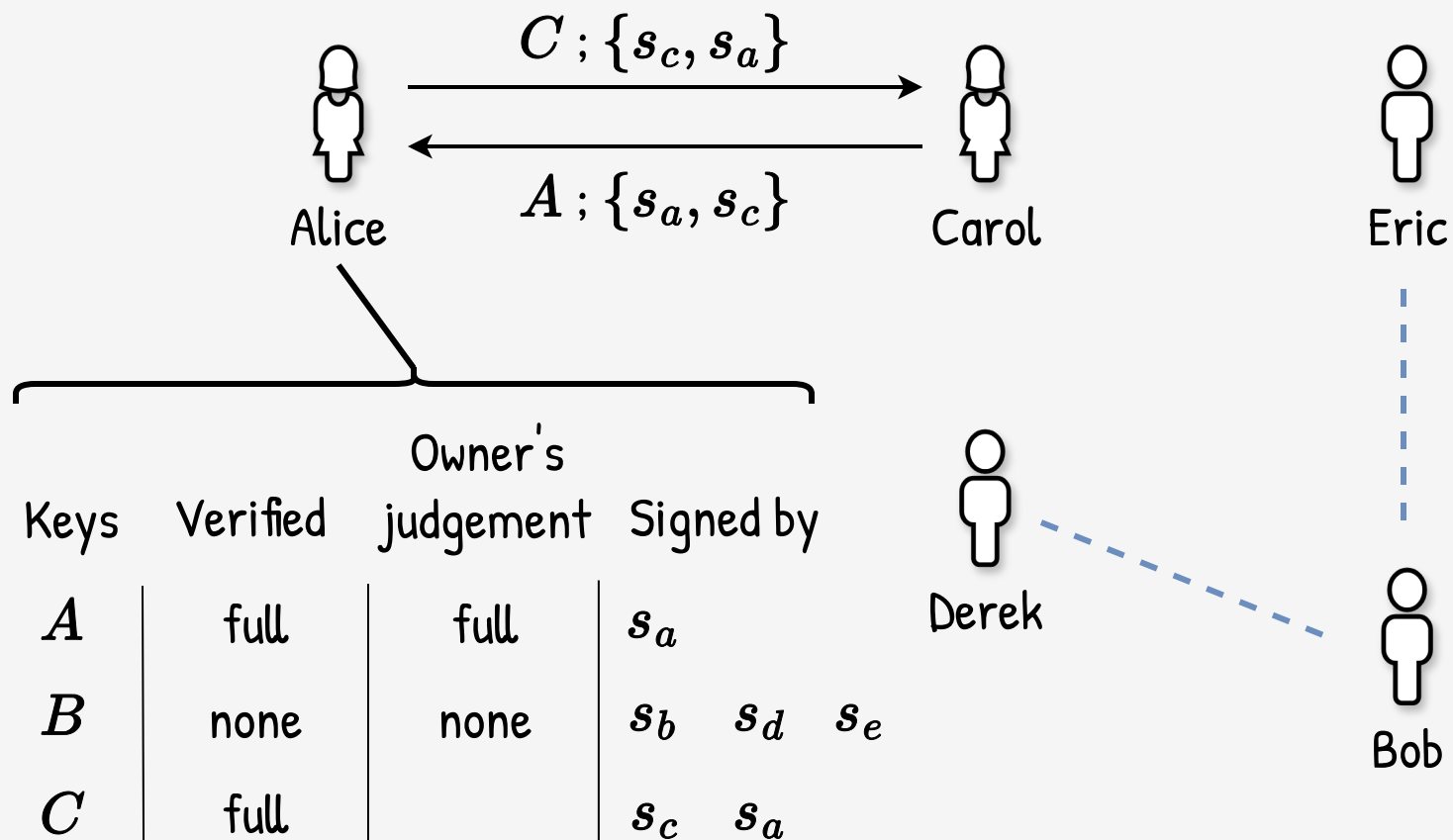
Eric

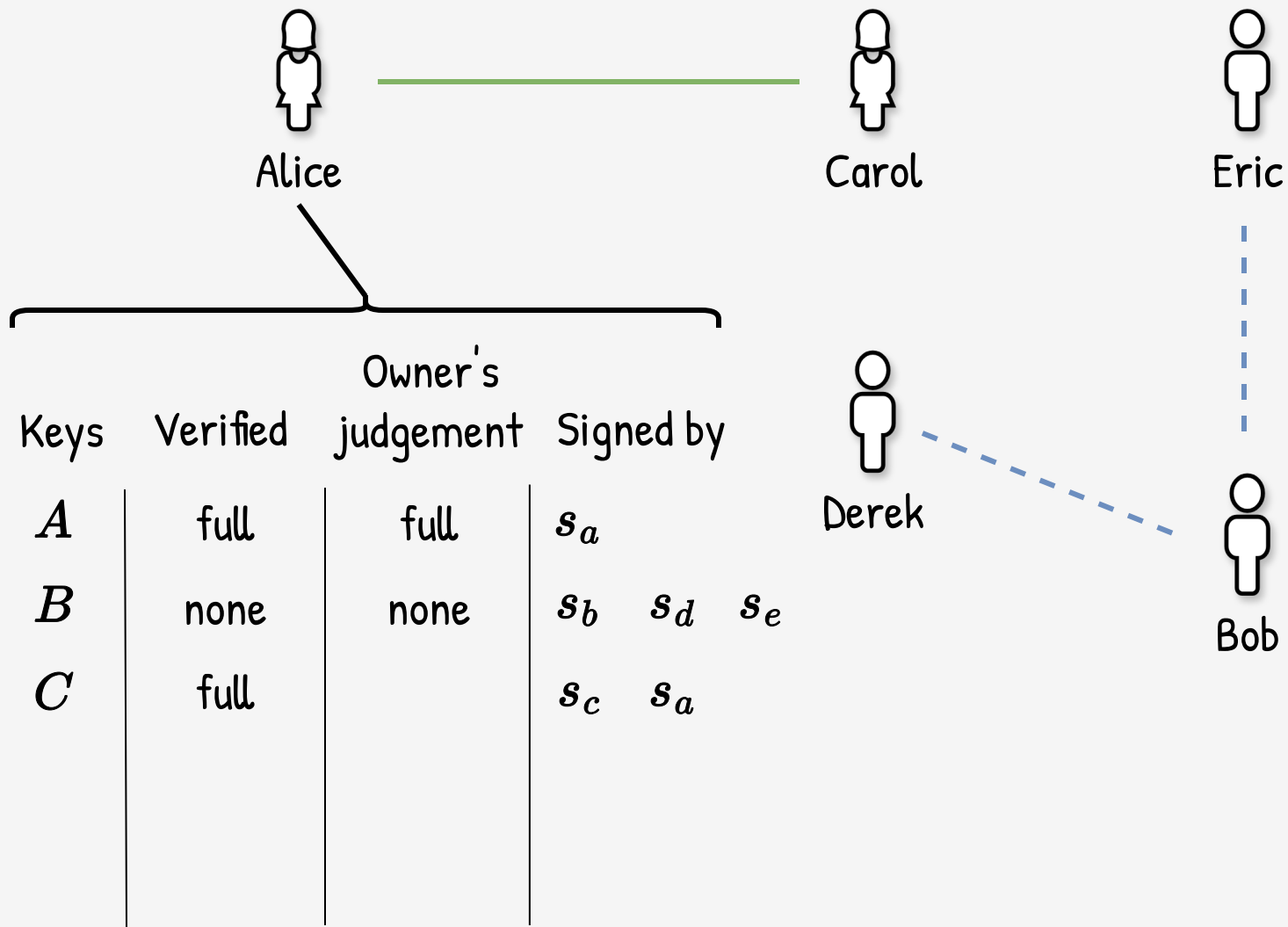
Owner's judgement					
Keys	Verified			Signed by	
<i>A</i>	full	full		s_a	
<i>B</i>	none	none		s_b	s_d s_e
<i>C</i>	full			s_c	s_a

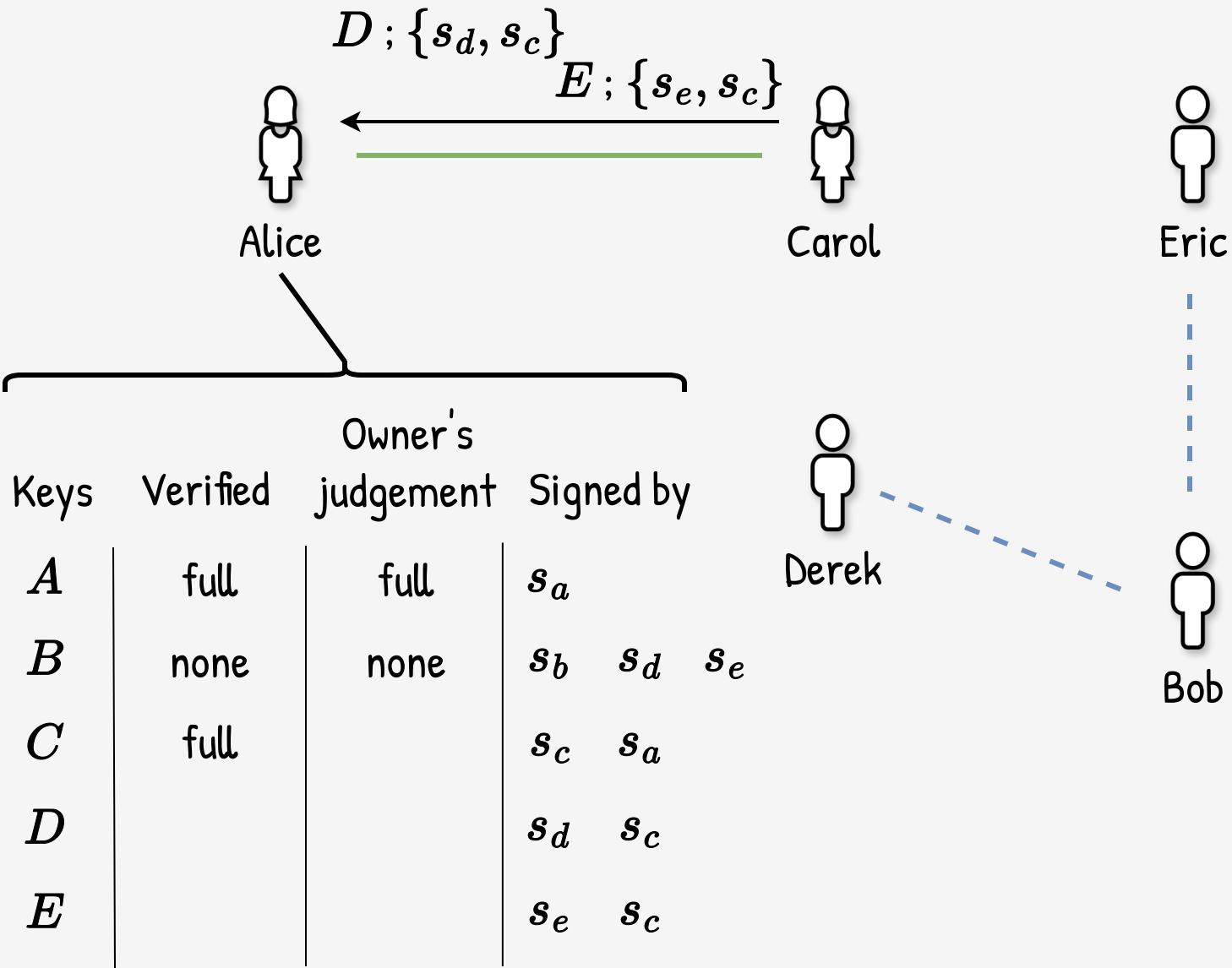
Derek

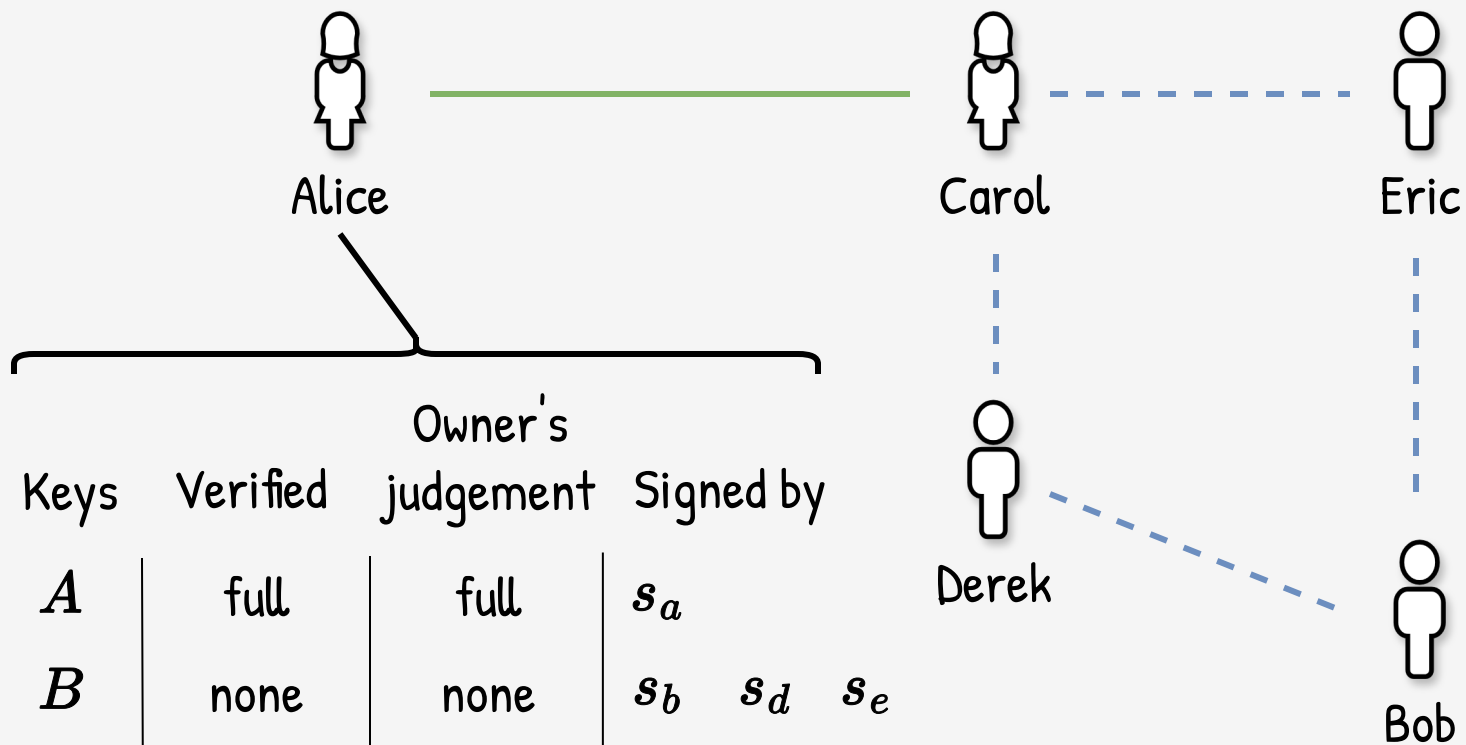
Bob

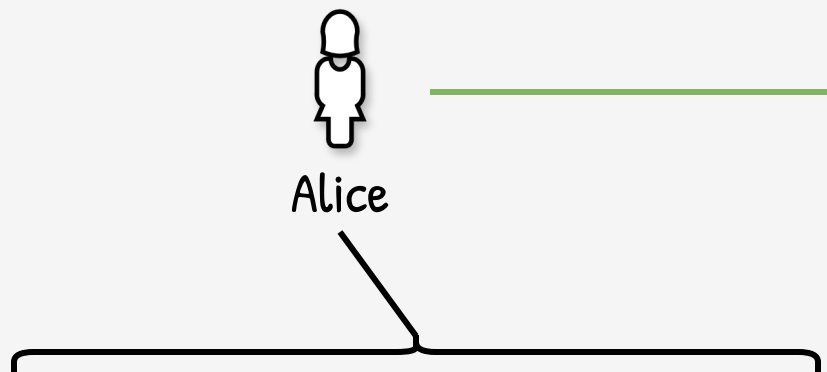
Alice sign *C* because she verified for real *C* belongs to Carol







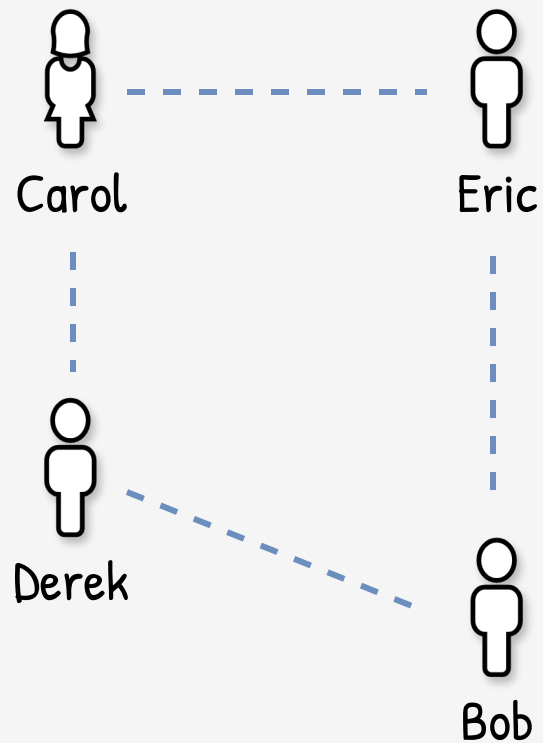




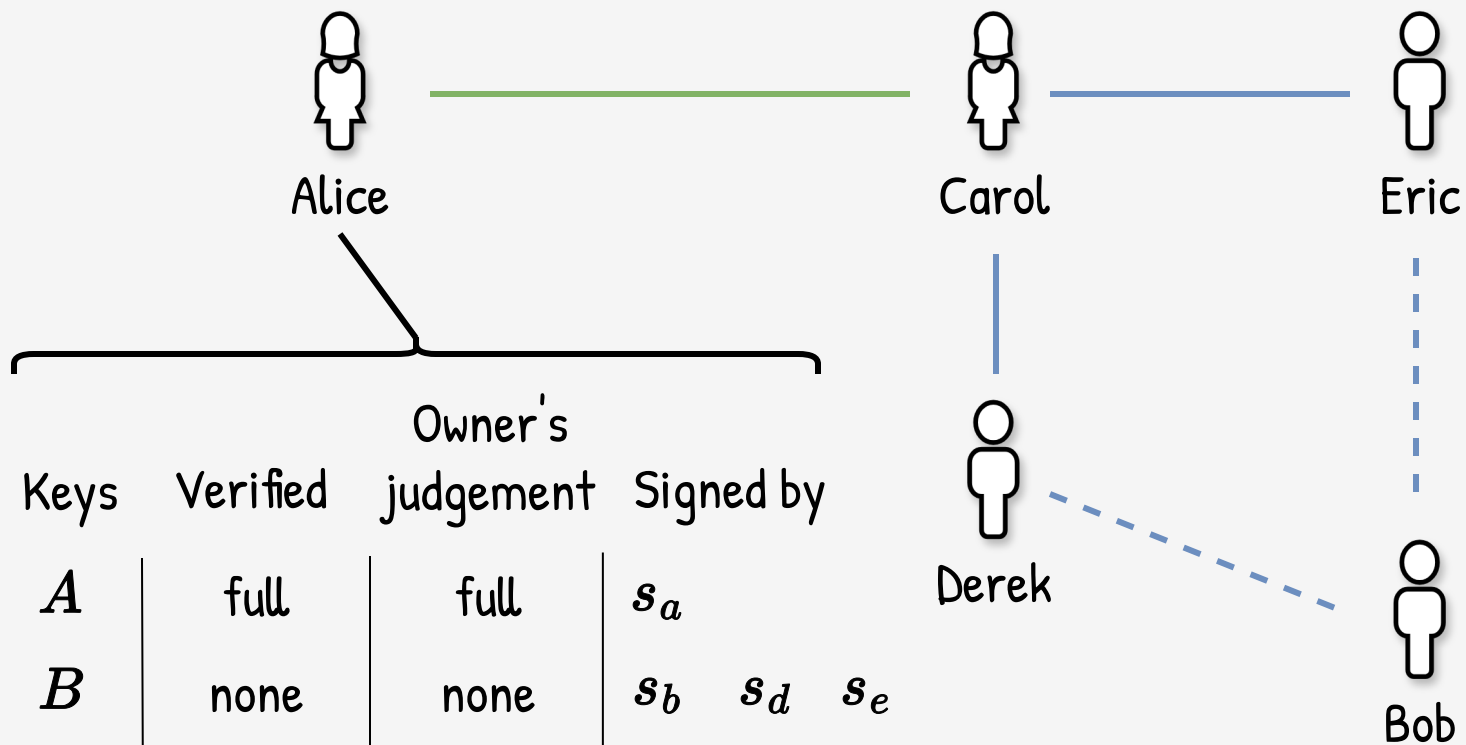
Owner's

judgement

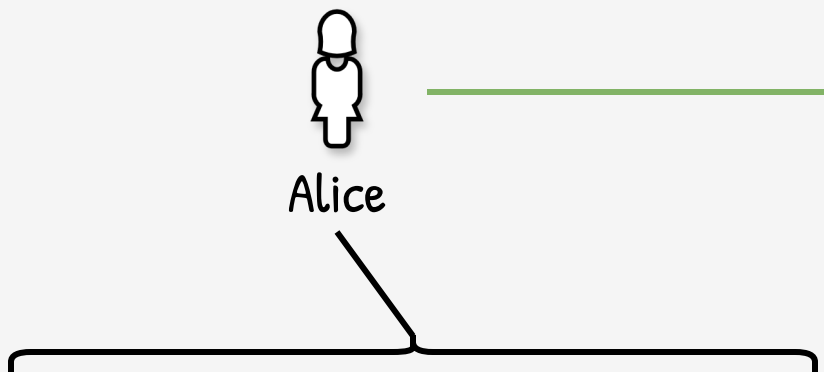
Keys	Verified	Signed by
A	full	s_a
B	none	s_b s_d s_e
C	full	s_c s_a
D	marginal	s_a s_c
E	marginal	s_e s_c



Because Alice thinks Carol is rigorous verifying, her signature marginally verifies D and E belong to Derek and Eric



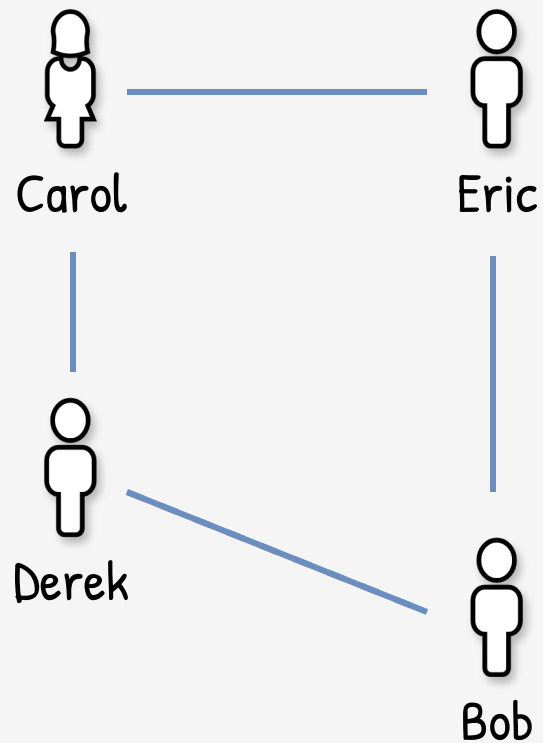
Keys	Verified	Owner's judgement	Signed by	
<i>A</i>	full	full	s_a	
<i>B</i>	none	none	s_b	$s_d \quad s_e$
<i>C</i>	full	full	s_c	s_a
<i>D</i>	marginal		s_d	s_c
<i>E</i>	marginal		s_e	s_c



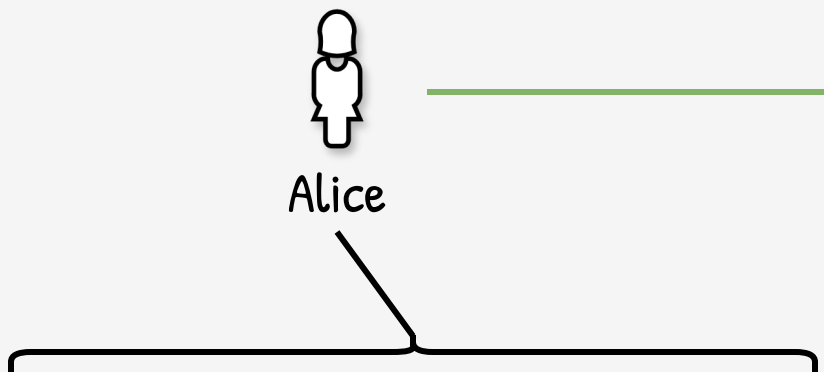
Owner's

judgement Signed by

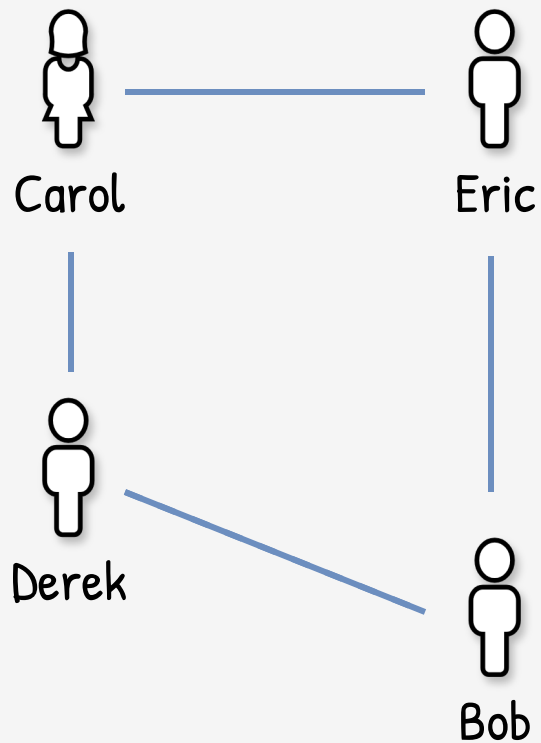
Keys	Verified	judgement	Signed by
A	full	full	s_a
B	marginal	none	s_d s_e
C	full	full	s_c s_a
D	marginal	marginal	s_a s_c
E	marginal	marginal	s_c s_e



Alice *marginally verifies* that B belongs to Bob because of Derek's and Eric's verifications



Keys	Verified	Owner's judgement	Signed by	
<i>A</i>	full	full	s_a	
<i>B</i>	marginal	none	s_b	$s_d \quad s_e$
<i>C</i>	full	full	s_c	s_a
<i>D</i>	marginal	marginal	s_d	s_c
<i>E</i>	marginal	marginal	s_e	s_c



Web of Trust

Takeaways

Symmetric
Cryptography

Fast

Easy



But relies on a
secret shared k so

Asymmetric
Cryptography

Slow

Not trivial!



It enables more
primitives

Signatures

Encryption

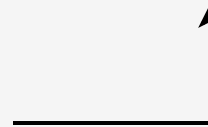
Certificates

Web of Trust

OK but with
weak points

But still, id verification
and pub key trust is

unresolved



And much more